

Spill-Cost-Aware Liveness Shaping for NPU Intra-Kernel Scheduling

Anonymous Author(s)

Abstract

Deep-learning compilers lower neural operators into kernel-level DAGs whose nodes mix micro-operations, short-lived tensors, and heterogeneous execution pipes. Under tight on-chip cache capacity, two legal topological orders can have similar peak pressure yet induce very different off-chip spill traffic. This paper identifies a structural asymmetry that standard schedulers often miss: clean buffers loaded from off-chip memory already have a backing copy and need not be written back when evicted, whereas dirty buffers produced by computation must be written before they are later reloaded. The two classes consume identical on-chip capacity but differ by a factor of two in spill cost. We introduce spill-cost-aware liveness shaping, which changes the legal schedule order to control the clean/dirty composition exposed inside capacity-pressure windows. The method keeps low-cost clean bytes available as eviction reserve and thereby reduces the cost of unavoidable spills. We prove a one-sided certificate for spill inevitability, give a linear-time overflow-area surrogate with a conditional constant-factor connection to optimal spill traffic, and explain why schedule order dominates victim-rule tuning under a Belady-margin condition. Experiments on public NPU-style scheduling instances, synthetic DAG distributions, small-graph oracles, and controlled clean/dirty ablations show that clean/dirty-blind pressure schedulers can pay 2.4–26× more spill traffic in capacity-bound regimes.

Keywords: kernel scheduling; neural network processor; DAG scheduling; cache residency; compiler optimization

1 Introduction

NPU intra-kernel scheduling maps hundreds to tens of thousands of micro-operations from an operator-level computation graph onto a finite on-chip cache hierarchy. It sits at the lowest level of the compiler stacks that lower

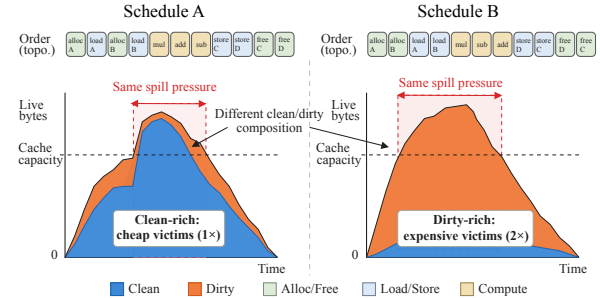


Figure 1. Spill-cost-aware liveness shaping. Two legal schedules can have similar capacity pressure while exposing different clean/dirty compositions to the eviction process. Keeping clean buffers resident in high-pressure windows provides low-cost eviction reserve and reduces off-chip traffic.

neural operators through successive graph- and IR-level optimizations [3, 10]. A schedule must respect data dependencies while managing the residency and eviction of intermediate values in scratchpad-like memories. When the live working set exceeds capacity, the compiler must temporarily spill some buffers to off-chip memory and reload them later. Such spills add data movement and can also block execution pipes.

Existing intra-kernel schedulers typically prioritize critical paths, resource pressure, or peak live bytes, treating spills as a local consequence of capacity shortage. This view misses a structural lever: once capacity overflow is unavoidable, different legal topological orders can expose markedly different buffer compositions inside the overflow window, and therefore incur substantially different eviction costs.

The key distinction is between clean and dirty kernel buffers. A clean buffer is read from off-chip memory and already has a backing copy; evicting it requires only a later reload. A dirty buffer is produced by computation; if it will be used again, eviction requires both a write-back and a reload. Clean and dirty buffers consume the same on-chip capacity, yet dirty eviction costs exactly 2× as much under the traffic model. A scheduler that only lowers the peak, or only changes victims on a fixed order, cannot fully exploit this asymmetry. We call the scheduling freedom that controls the clean/dirty composition of overflow windows *spill-cost-aware liveness shaping*. Two facts behind it are each known on their own: schedule order affects liveness, as in instruction scheduling, and clean victims are cheaper to evict, as in register allocation and cache replacement.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CGO '27,

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

What is new here is treating their interaction as a degree of freedom the scheduler can actively shape in a kernel-level DAG. We defer a detailed contrast with those lines of work to Section 5.

This paper makes three contributions:

1. We formulate spill-cost-aware liveness shaping for NPU intra-kernel DAGs. The method changes schedule order to control the clean/dirty residency composition in overflow windows, preserving low-cost eviction reserve when capacity is tight.
2. We give a theoretical characterization of the regime in which this matters. The spill-inevitability certificate in Theorem 3.1 provides a one-sided, linear-time diagnostic for identifying cases where spills cannot be avoided by any near-optimal schedule. The Belady-margin stability result in Proposition 3.3 explains why schedule order, rather than victim-rule tuning, is the main optimization degree of freedom. The overflow-area theorem in Theorem 3.2 connects a cheap liveness surrogate to optimal spill traffic within constant factors under bounded absence durations, explaining when the surrogate is expected to track real traffic.
3. We evaluate the method across public NPU-style benchmark instances, synthetic DAG distributions, small-graph oracles, and controlled ablations. The results show that benefits concentrate exactly in capacity-bound regions and that clean/dirty-blind pressure schedulers can pay 2.4–26× more spill traffic.

2 Problem Model

We study DAGs and cache parameters from a public NPU intra-kernel scheduling benchmark that models compiler-generated kernel schedules. The input is a micro-operation graph $G = (V, E)$ for a neural operator. Its nodes have two types, illustrated in Figure 2: *operation nodes*, which execute computation or data movement and carry execution units, latencies, and read/write buffer sets; and *cache-management nodes*, which mark the beginning and end of buffer lifetimes. Each logical buffer b has a unique allocation node a_b and free node f_b , a size s_b , and a cache type $\tau(b) \in \mathcal{C}$. Each cache $c \in \mathcal{C}$ has an abstract capacity Cap_c ; the concrete values are given in Section 4. A *schedule* $S = (v_1, \dots, v_N)$ is a topological order of all required nodes.

A buffer b is *clean* if it is written by an operation that reads data from off-chip memory; in this case, off-chip storage already contains a backing copy, so eviction needs no write-back and contributes only a reload cost $e_b = s_b$. Otherwise b is *dirty*: evicting it requires a write-back and a later reload, for extra traffic $e_b = 2s_b$. The two classes occupy identical on-chip space yet differ by 2× in spill cost—the core asymmetry our method exploits.

We evaluate schedule quality through three nested views. **P1** considers only node order and minimizes per-cache

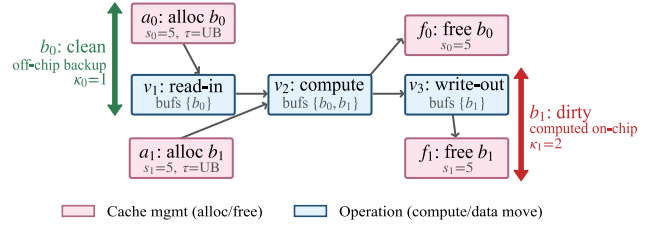


Figure 2. A micro-operation DAG. Pink nodes are cache-management events (allocation/free); blue nodes are operations. Buffer b_0 is loaded from off-chip memory and already has a backing copy, so it is *clean* ($\kappa = 1$). Buffer b_1 is produced by computation, so it is *dirty* ($\kappa = 2$).

peak live residency. **P2** adds physical address assignment and spill insertion, minimizing total extra traffic

$$E(S) = \sum_{b \in \text{Spills}} e_b.$$

P3 simulates the earliest completion time under dependencies and pipe constraints for a fixed order. All three views share the same asymmetric clean/dirty traffic model. Therefore, an order that keeps more clean reserve in overflow windows can improve residency, traffic, and timing behavior at once.

3 Method

Existing schedulers treat spills as a local consequence after capacity is exceeded and then apply a victim rule to a fixed order. Our experiments show that this is the wrong primary lever: on a fixed order, four Belady-style victim variants usually differ by at most 4% in extra traffic, whereas legal schedule orders can vary by more than an order of magnitude (Section 4.3). Our method therefore optimizes schedule order to shape the clean/dirty composition of overflow windows.

The method has three stages. First, spill diagnosis (Section 3.1) asks whether spills are unavoidable for the given DAG and capacity. If they are, the objective shifts from eliminating spills to lowering their cost, and overflow area becomes a cheap cross-stage surrogate. Second, candidate-order generation and address assignment (Section 3.2) produce complementary topological orders and run best-fit placement with cost-aware spill insertion (Algorithm 1). Third, we simulate the Cartesian product of candidate orders and prefetch windows, then choose the best combination by a lexicographic objective (Figure 3).

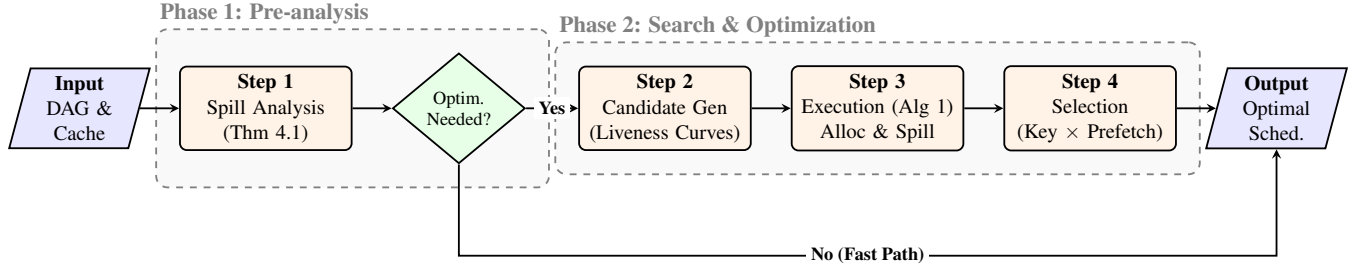


Figure 3. Method overview. The optimization framework consists of three stages: (1) inevitable spill determination to decide if optimization is needed; (2) three complementary candidate topological orders generated via liveness shaping; (3) Algorithm 1 which executes address assignment and spill insertion along each order, followed by lexicographic key selection across candidate orders and prefetch windows.

3.1 Spill Diagnosis and Surrogate Objective

If some schedule keeps every cache peak within capacity, then no spill is needed and the clean/dirty distinction is irrelevant. The first question is therefore: *for this DAG and capacity, are spills unavoidable for all reasonable schedules?* We use a linear-time certificate. Let $\mathcal{O}_\epsilon$ denote schedules whose extra traffic is within $(1 + \epsilon)$ of optimum, with $\epsilon = 0.1$. Let $\underline{W}_c^\epsilon = \min_{S \in \mathcal{O}_\epsilon} \max_t W_c^S(t)$ be the best per-cache peak working set within that near-optimal set.

Theorem 3.1 (Spill-inevitability certificate). *If $\underline{W}_c^\epsilon > \text{Cap}_c$, then every schedule in $\mathcal{O}_\epsilon$ must spill on cache c . For any feasible implementation P of a schedule S , the extra traffic satisfies $\text{extra}(P) \geq \max_t (W_c^S(t) - \text{Cap}_c)_+$. The proof is in the supplementary material.*

This certificate is deliberately one-sided: $\underline{W}_c^\epsilon > \text{Cap}_c$ confirms that spills are unavoidable, but $\underline{W}_c^\epsilon \leq \text{Cap}_c$ does not prove that a spill-free implementation exists. We use it as an applicability diagnostic rather than a complete feasibility test.

Peak live bytes are a direct way to compare candidate orders, but a P1-optimal order can exceed capacity severely on a particular cache and produce high P2 traffic. We therefore use overflow area as the scheduling surrogate:

$$A(S) = \sum_t \sum_{c \in \mathcal{C}} (\text{live}_c(t) - \text{Cap}_c)_+. \quad (1)$$

In the implementation we use the normalized score $\Phi(S) = \sum_t \sum_c (\text{live}_c(t) - \text{Cap}_c)_+ / \text{Cap}_c$. The next result connects this cheap surrogate to optimal spill traffic under bounded absence durations.

Theorem 3.2 (Conditional overflow-area approximation). *Assume every spilled interval has absence duration at most L . Suppose there exists a feasible plan whose removed area satisfies $R(P) \leq \lambda A(S)$ and whose spilled intervals have absence duration at least ℓ . If unit costs obey $\kappa \in [1, 2]$, then*

$$\frac{1}{L} A(S) \leq E^*(S) \leq \frac{2\lambda}{\ell} A(S). \quad (2)$$

The proof and a matching counterexample are in the supplementary material.

The surrogate Φ is useful for three reasons: it costs $O(N)$ to compute for each candidate order, it gives a single scalar used consistently across P1, P2, and P3, and Theorem 3.2 supplies a theoretical explanation for why it should track spill traffic in capacity-bound regimes. It is not a global optimality guarantee for schedule selection.

3.2 Candidate Orders and Spill Insertion

The candidate-order design is structure-agnostic. Orders depend only on buffer clean/dirty state, size, and cache capacity; they do not encode operator-specific motifs. The candidates share a memory-aware list-scheduling skeleton with three ready sets: frees, operations, and allocations. Ready frees are scheduled first to reduce residency pressure. Ready allocations are delayed until all ready frees and operations have been scheduled, which compresses allocation-to-free distance. We instantiate three complementary orders:

- (1) **Pressure-aware order.** Allocation nodes are ranked by successor indegree. A smaller indegree indicates that consumers will become ready soon, so the allocation can be delayed until it is close to use.
- (2) **Capacity-throttled order.** Allocation nodes are ranked by identifier and overlaid with capacity gating: an allocation that would push a cache over capacity is held back until no further delay is possible, with capacity weights normalized by $1/\text{Cap}_c$ to balance caches of differing tightness.
- (3) **ID-reserve order.** Within the three node classes, this order chooses the minimum identifier and performs no capacity throttling. It is a method candidate, not an external baseline: its purpose is to preserve the stable input-buffer order that often keeps clean COPY-IN buffers resident through compute windows, leaving cheap eviction reserve in overflow regions.

Given a schedule order, Algorithm 1 performs address assignment and spill insertion along the order. When capacity

is insufficient, it selects an eviction victim by

$$\text{victim} = \arg \max_b d(b) \cdot \frac{s_b}{e_b} = \arg \max_b \begin{cases} d(b), & b \text{ clean,} \\ d(b)/2, & b \text{ dirty,} \end{cases} \quad (3)$$

where $d(b)$ is the next-use distance. Dirty buffers have their effective distance halved, so the engine prefers cheap clean evictions when future-use distances are comparable.

Proposition 3.3 (Belady-margin stability). *Consider distance-dominant scoring rules whose cost-factor ratio satisfies $\rho = g_{\max}/g_{\min} \leq 2$. If the minimum next-use distance among selected victims is greater than ρ times the maximum next-use distance among non-victims, then all such rules select the same victim set. The proof is in the supplementary material.*

Thus schedule order, not the victim rule, is the dominant axis on spill cost; we quantify both effects in Section 4.3.

Reload timing does not change spill traffic, but it does affect whether reload latency can overlap with computation. The prefetch window H in Algorithm 1 controls this dimension: a spilled buffer whose next use is within H steps is reloaded early if free space is available and no extra eviction is needed. For P2 we use $H \in \{0, 5, 40\}$ and select by (E, n, T) . For P3 we use $H \in \{0, 5, 40, 80, 120\}$ and select by (T, E, n) . The candidate set is monotone: adding a new order or prefetch window can only improve or preserve the selected objective, because the final choice uses the true P2/P3 key rather than the surrogate (see the supplementary material).

Complexity. The overall method runs in $O(K \cdot W \cdot (N \log N + E + B^2))$ time, where $K = 3$ is the number of candidate orders, W is the prefetch-window grid size (3 for P2, 5 for P3), N is the number of DAG nodes, and B is the number of buffers. Candidate-order generation costs $O(N \log N + E)$ (topological sort with a priority queue). The bottleneck of address assignment is Belady victim selection: in the worst case each allocation scans all resident buffers, giving $O(B)$ per allocation over B allocations, hence $O(B^2)$. The pipeline simulation is $O(N + E)$ in the number of edges E ; on the sparse DAGs of our benchmark this term is below the combinatorial search and spill-assignment terms.

4 Experiments

The experiments test the paper’s central claim: when capacity overflow is unavoidable, can schedule order reduce real spill cost by changing the clean/dirty composition inside peak-pressure windows? We build evidence at four levels: controlled reorder comparisons on public NPU-style benchmark instances, mechanism decomposition with an applicability analysis, exact small-graph oracle checks for the theoretical bounds, and clean/dirty cost ablations that isolate the $2\times$ asymmetry. The four levels keep the conclusion from resting on a single black-box comparison.

Algorithm 1 Address assignment and spill insertion

Require: Topological order S , capacities $\{\text{Cap}_c\}$, prefetch window H

Ensure: Extended order, address assignment, spill list

- 1: Initialize a best-fit free-list manager F_c for each cache c
- 2: **for** $(t, v) \in \text{enumerate}(S)$ **do**
- 3: **if** $v = \text{ALLOC}(b)$ **then**
- 4: $\text{offset} \leftarrow \text{PLACE}(b, t)$
- 5: $\text{memory}[b] \leftarrow \text{offset}$
- 6: **else if** $v = \text{FREE}(b)$ **then**
- 7: **if** b is spilled **then** $\text{RELOAD}(b, t)$
- 8: **end if**
- 9: $F_{\tau(b)}.\text{Release}(\text{offset}_b, s_b)$
- 10: **else**
- 11: **for** $b \in v.\text{bufs}$ where b is spilled **do**
- 12: $\text{RELOAD}(b, t)$
- 13: **end for**
- 14: **for** $b \in \text{SpilledSet}$ **do**
- 15: **if** $d(b) - t \leq H$ and $F_{\tau(b)}$ has $\geq s_b$ free **then**
- 16: $\text{RELOAD}(b, t)$
- 17: **end if**
- 18: **end for**
- 19: **end if**
- 20: **end for**
- 21: **procedure** $\text{PLACE}(b, t)$
- 22: $\text{offset} \leftarrow F_{\tau(b)}.\text{BestFit}(s_b)$
- 23: **while** $\text{offset} = \text{nil}$ **do**
- 24: $v^* \leftarrow \arg \max_{b' \in \text{Resident}_{\tau(b)}} d(b')s_{b'}/e_{b'}$
- 25: $\text{emit SPILLOUT}(v^*)$; $\text{release}(\text{offset}_{v^*}, s_{v^*})$
- 26: $\text{offset} \leftarrow F_{\tau(b)}.\text{BestFit}(s_b)$
- 27: **end while**
- 28: $F_{\tau(b)}.\text{Allocate}(\text{offset}, s_b)$
- 29: **return** offset
- 30: **end procedure**

4.1 Experimental Setup

We evaluate on a set of public NPU intra-kernel scheduling instances built from compiler-style micro-operation DAGs. They cover three mainstream operator families—convolution, FlashAttention, and matrix multiplication—with graph sizes from about 1.7×10^3 to 3.6×10^4 nodes, large enough to exhibit the capacity-bound behavior typical of kernel-level scheduling (Table 1). The hardware abstraction has five on-chip caches with abstract capacities $L1 = 4,096$, $UB = 1,024$, $L0A = 256$, $L0B = 256$, and $L0C = 512$. The instances are used only for evaluation; our method hard-codes nothing specific to any operator structure.

Table 1. Benchmark instances. FA denotes FlashAttention.

Instance	Op. type	$ V $	$ E $	Buffers
Conv_Case0	Conv	2,580	3,869	831
Conv_Case1	Conv	36,086	85,653	12,013
FA_Case0	FA	1,716	2,712	572
FA_Case1	FA	6,952	11,184	2,328
Matmul_Case0	Matmul	4,160	7,104	1,216
Matmul_Case1	Matmul	30,976	55,040	8,960

Table 2 lists the four evidence layers. The common thread is a single *same-engine* protocol: every comparator and our method share one best-fit address-assignment and spill engine (Algorithm 1) and only the input topological order is swapped. This attributes spill-traffic differences cleanly to how order shapes clean/dirty residency, not to differences in the eviction implementation. We report the three views defined in Section 2—P1 per-cache peak residency, P2 total spill traffic $E(S)$, and P3 pipelined execution time—and take P2 as the primary view because it directly measures the spill cost targeted by liveness shaping.

Table 2. The four evidence layers used in the evaluation.

Layer	Scale	Role
Public NPU cases	6 DAGs	Same-engine reorder comparison on compiler-style instances.
Synthetic suite	36 DAGs	Separate capacity-bound from order-reachable regimes.
CP-SAT oracle	8 graphs	Fixed-order optimum and theorem bounds.
Clean/dirty ablation	GEMM	Isolate the 2× reserve-cost mechanism.

The main controls are drawn from the classic latency-oriented and memory-pressure-oriented paradigms, all of which are blind to the clean/dirty distinction. Critical-path list scheduling [9, 15] minimizes critical-path latency and ignores memory pressure. A uniform-pressure greedy minimizes the stepwise peak of live bytes but treats clean and dirty buffers identically. A Goodman–Hsu-style latency/pressure mode switch [6] alternates between critical-path priority and pressure reduction, representing the classic integrated-prepass idea. Beyond these, we add a *free-first critical-path companion* as a stronger control: it keeps the critical-path order but schedules ready frees early to shorten live ranges, making the order more amenable to the allocator while still carrying no explicit cost awareness. It tests how much of our benefit “free early, allocate late” alone can explain.

Our solver keeps a practical runtime even on the largest instance (Table 3). P1 emits a single schedule and finishes within 0.2 s on every instance. P2 and P3 simulate the Cartesian product of candidate orders and prefetch windows, so their cost grows with instance size, reaching about 73 s for P3 on the largest instance, Conv_Case1 (median of three repetitions).

Table 3. Solver wall-clock runtime (seconds; median of three repetitions).

Instance	P1	P2	P3
Conv_0	0.009	0.391	0.683
Conv_1	0.189	57.971	73.141
FA_0	0.006	0.220	0.328
FA_1	0.027	2.723	3.356
Matmul_0	0.014	0.701	1.137
Matmul_1	0.126	19.402	32.328

4.2 Main Baseline Results

Figure 4 and Table 4 report P2 spill traffic for the six public benchmark cases under the same spill engine. The memory-aware but clean/dirty-blind uniform-pressure and Goodman–Hsu orders are substantially worse than our method on every case, paying 2.4–26× more extra traffic, with a median gap of about 11×. The pure latency-oriented critical-path order is further behind, by roughly 8–54×. This systematic gap shows that minimizing total live bytes or critical-path latency is not enough to approach the true spill-cost lower bound: the key is to shape the eviction reserve in peak windows toward cheaper clean bytes. Across all 6 cases, 3 views, and 4 comparators (72 combinations), our order attains a lower or equal objective in every one.

The free-first critical-path companion is a strong baseline. Except on Conv_Case0, where it trails by 2.4×, its median P2 gap from our method is only about 1%, and it ties on FA_Case0 and Matmul_Case1. This indicates that an allocator-friendly latency order already reduces spills indirectly by shortening live ranges, capturing part of the benefit. However, because it has no explicit clean/dirty cost shaping, it remains systematically behind on cases such as Conv_Case0 where clean eviction reserve is decisive for cost, and it is consistently dominated by our method across the multi-objective evaluation.

4.3 Mechanism and Applicability

The previous comparison shows a systematic spill-traffic gap; we now ask where the benefit comes from and where it applies.

Schedule order versus victim rule. With a fixed order, four Belady-style victim variants produce identical extra traffic on Matmul and FA instances (coefficient of variation $CV = 0$) and at most about 4% CV on Conv rows; across

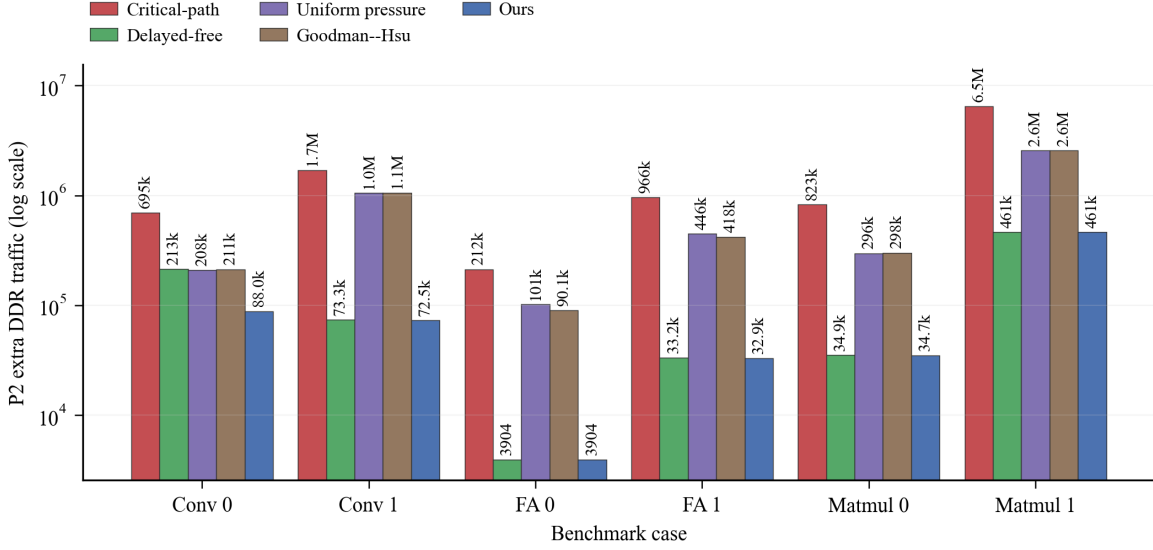


Figure 4. Standard scheduler comparison for P2 spill traffic on a log scale. All orders use the same address-assignment and spill engine; only the topological order changes.

Table 4. P2 spill traffic $E(S)$ under a shared spill engine. FA denotes FlashAttention; the best value in each row is bold.

Instance	CP list	Pressure	G-Hsu	CP-free	Ours
Conv_0	694,506	207,932	210,844	212,956	88,044
Conv_1	1,695,264	1,048,524	1,053,484	73,348	72,520
FA_0	211,784	101,356	90,092	3,904	3,904
FA_1	965,944	445,876	418,228	33,152	32,920
Matmul_0	823,168	296,064	298,240	34,944	34,688
Matmul_1	6,506,240	2,556,928	2,560,640	460,800	460,800

different legal orders under the same engine, extra traffic can swing by more than 10×. Figure 5 decomposes an L1 high-pressure window from a benchmark case into clean and dirty residency: under the same capacity, two legal orders expose markedly different peak heights and dirty fractions. This confirms the prediction of Proposition 3.3: when next-use distance dominates, the victim rule has far smaller marginal effect than schedule order, which is the main lever on spill cost.

Applicability domain. A systematic evaluation on a synthetic DAG distribution sharpens the boundary. In the regions marked capacity-bound by the certificate of Theorem 3.1, our method wins against both critical-path list scheduling and random orders in 100% of cases, and against the free-first companion in 77.8% of cases, with a median 2× advantage. In the order-reachable subgroup, a good order can avoid spills directly, leaving no room for clean/dirty shaping, and the win rate over the free-first companion drops to 0%. This matches the certificate’s role: it marks the regime in which the cost composition of unavoidable spills

must be optimized, and the method’s effectiveness tracks that precondition.

Theory checks and surrogate behavior. Small-graph CP-SAT oracles verify two conclusions. For the selected order, our engine reaches the capacity-only optimal extra traffic (ratio 1.0). The lower bound of Theorem 3.1 holds on all small graphs, and the empirical ratios for Theorem 3.2 satisfy $E^*/A \in [0.56, 1.13]$, inside a constant-factor band. The overflow-area surrogate Φ has almost the same global Spearman correlation with extra traffic as per-cache peak pressure (0.958 versus 0.955). The value of Φ is not a sizable accuracy gain but its $O(N)$ cost, which makes full candidate evaluation feasible while Theorem 3.2 gives it a consistent cross-stage justification. Per-instance figures and data are in the supplementary material.

Controlled ablation. To rule out instance-specific artifacts, we build synthetic GEMM kernels that fix the same DAG structure, number of spills, and peak, and change only the type of reserve inside the peak window. Clean reserve incurs 1,536 units of extra traffic and dirty reserve incurs

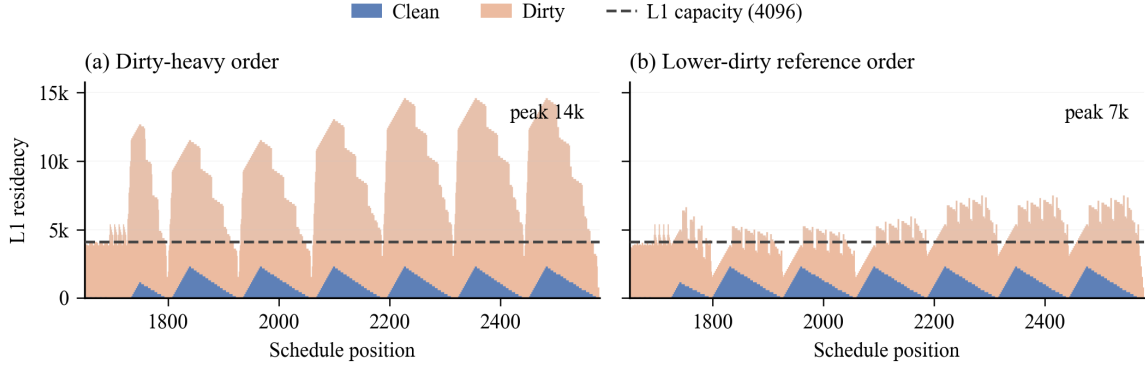


Figure 5. Clean/dirty residency decomposition in an L1 high-pressure window. Blue is clean residency that already has an off-chip copy; orange is dirty residency requiring write-back; the dashed line is L1 capacity. Under the same capacity, different legal orders expose different peak heights and dirty composition.

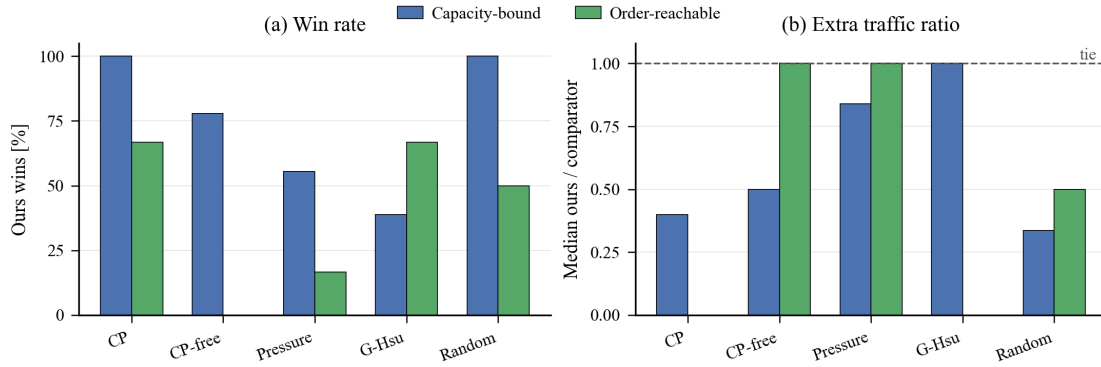


Figure 6. Applicability across synthetic regimes. Left: the win rate of our method against each comparator. Right: the median extra-traffic ratio (ours/comparator; lower is better), with the dashed line marking parity. The method is strongest in capacity-bound instances and naturally loses its systematic advantage in order-reachable instances, where a good order avoids spills outright.

3,072, exactly $2\times$; under full isolation this confirms the predicted clean/dirty cost asymmetry. A further schedule sweep shows that extra traffic decreases monotonically as the clean fraction at the peak rises.

4.4 Limitations

The applicability boundary should be stated precisely. If an instance is order-reachable—some legal order keeps every cache peak within capacity—then no spill is needed and clean/dirty shaping has nothing to act on; and if a conventional pressure/latency hybrid already preserves enough clean reserve, our advantage shrinks or vanishes. The approximation in Theorem 3.2 also relies on bounded absence durations, so workloads dominated by long-lived tiny spills can weaken the surrogate. Finally, we target a single static kernel DAG fully known at compile time; the extension to multiple cores, dynamic control flow, and more complex memory hierarchies is left to future work.

5 Related Work

Joint scheduling and memory optimization has a long history in DNN accelerator compilation. COSMA [11], for example, observes that non-compulsory off-chip accesses depend jointly on operator schedule, memory allocation, and tensor replacement, and optimizes the three with an ILP. That line of work solves a graph-level placement and replacement problem. In contrast, NPU intra-kernel scratch-pad scheduling operates at a much finer micro-operation granularity: clean and dirty buffer states introduce a $2\times$ spill-cost asymmetry, and schedule order determines not only the live byte peak but also the cost composition of the buffers exposed to eviction.

Compiler back-ends have also long recognized the coupling between scheduling and live ranges. Goodman-Hsu-style integrated prepass scheduling [6] and later register-pressure-aware schedulers [13, 14] trade instruction-level parallelism against register pressure. Local allocators and

replacement heuristics exploit the fact that clean values can be evicted without a store [7, 12]. Caching theory generalizes Belady's farthest-future rule [1] to nonuniform-size and nonuniform-cost objects [5]. These results show that "schedule affects liveness" and "clean victims are cheaper" are not new in isolation. The additional freedom in intra-kernel DAG scheduling is that the request order itself can be changed: a compiler can actively shape live-range overlap and clean/dirty composition rather than only choose victims for a fixed stream.

Rematerialization and activation checkpointing reduce memory pressure from a different direction [4, 8]; recent spill-free compilation [2] aims to eliminate spills by shortening live ranges. These techniques are complementary. In large kernel-level NPU schedules, capacity constraints often make some spills unavoidable. In that regime, reloadable input buffers form a natural clean reserve, and the central question is how schedule order can reduce the write-back cost of unavoidable evictions.

6 Conclusion

This paper identifies a structural lever in NPU intra-kernel scheduling: the $2\times$ asymmetry between clean and dirty spill costs means that schedule order controls not only peak live bytes, but also the cost composition of the buffers available for eviction in capacity-pressure windows. When spills are unavoidable, keeping more cheaply evictable clean reserve substantially reduces real off-chip traffic. Around this finding we give a spill-inevitability certificate (Theorem 3.1), a conditional overflow-area approximation (Theorem 3.2), and a Belady-margin stability result (Proposition 3.3) that explains the empirical pattern of an insensitive victim rule alongside a highly sensitive schedule order.

Experiments support the claim at four levels. On the public benchmark cases, clean/dirty-blind memory-pressure schedulers pay $2.4\text{--}26\times$ more P2 spill traffic than our method (Section 4.2). The synthetic distribution shows that the gains concentrate in the capacity-bound region delimited by the certificate; in order-reachable instances a simple order already avoids spills and our method has no systematic advantage (Section 4.3). Small-graph CP-SAT oracles confirm fixed-order engine optimality and the empirical constant ranges of the two bounds, and the controlled ablation isolates the $2\times$ clean/dirty cost gap, showing that the benefit is not an artifact of particular instances.

Future work includes extending spill-cost-aware liveness shaping to multi-core scheduling and cross-core cache coherence; supporting graphs with dynamic control flow such as conditional branches and loops; and modeling cross-level spill cascades in deeper memory hierarchies.

References

- [1] L. A. Belady. 1966. A Study of Replacement Algorithms for a Virtual-Storage Computer. *IBM Systems Journal* 5, 2 (1966), 78–101.

- [2] Prasanth Chatarasi, Alex Gatea, Wei Wang, Chris Bowler, Shubham Jain, Masoud Ataei Jaliseh, Nicole Khoun, Alberto Mannari, Bardia Mahjour, Viji Srinivasan, and Swagath Venkataramani. 2026. Enabling Spill-Free Compilation via Affine-Based Live Range Reduction Optimization. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 1–13.
- [3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Carlsbad, CA, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [4] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *arXiv preprint abs/1604.06174* (2016).
- [5] Christian Coester, Roie Levin, Joseph (Seffi) Naor, and Ohad Talmon. 2022. Competitive Algorithms for Block-Aware Caching. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [6] James R. Goodman and Wei-Chung Hsu. 1988. Code Scheduling and Register Allocation in Large Basic Blocks. In *Proceedings of the 2nd International Conference on Supercomputing (ICS)*. 442–452.
- [7] Jia Guo, Maria Jesus Garzaran, and David Padua. 2004. The Power of Belady's Algorithm in Register Allocation for Long Basic Blocks. In *Languages and Compilers for Parallel Computing (LCPC) (Lecture Notes in Computer Science, Vol. 2958)*. Springer, 374–389. doi:10.1007/978-3-540-24644-2_24
- [8] Paras Jain, Ajay N. Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [9] Yu-Kwong Kwok and Ishfaq Ahmad. 1999. Static Scheduling Algorithms for Allocating Directed Task Graphs to Multiprocessors. *Comput. Surveys* 31, 4 (1999), 406–471.
- [10] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.1109/CGO51591.2021.9370308
- [11] Yi Li, Aarti Gupta, and Sharad Malik. 2023. Combined Scheduling, Memory Allocation and Tensor Replacement for Minimizing Off-Chip Data Accesses of DNN Accelerators. *arXiv preprint abs/2311.18246* (2023).
- [12] Vincenzo Liberatore, Martin Farach-Colton, and Ulrich Kremer. 1999. Evaluation of Algorithms for Local Register Allocation. In *Proceedings of the 8th International Conference on Compiler Construction (CC) (Lecture Notes in Computer Science, Vol. 1575)*. Springer, 137–153. doi:10.1007/978-3-540-49051-7_10
- [13] Ghassan Shobaki, Ahmad Al Qutami, and Najm Eldeen Abu-Rmaleh. 2022. Register-Pressure-Aware Instruction Scheduling Using Ant Colony Optimization. *ACM Transactions on Architecture and Code Optimization* 19, 4 (2022).
- [14] Ghassan Shobaki, Saeed Shawabkeh, and Najm Eldeen Abu-Rmaleh. 2013. Preallocation Instruction Scheduling with Register Pressure Minimization Using a Combinatorial Optimization Approach. *ACM Transactions on Architecture and Code Optimization* 10, 4 (2013).
- [15] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-effective and Low-complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274. doi:10.1109/71.993206